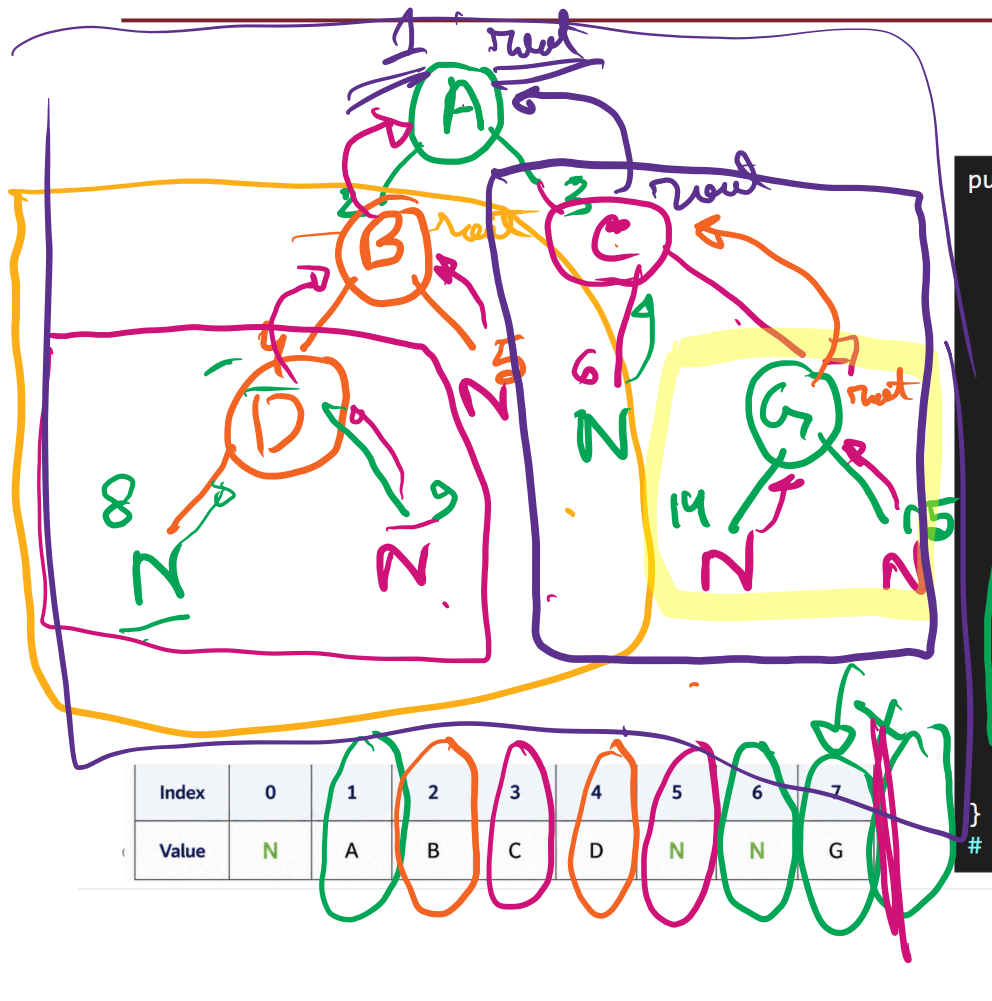
Recursive Function: arrToTree() (Show simulation)

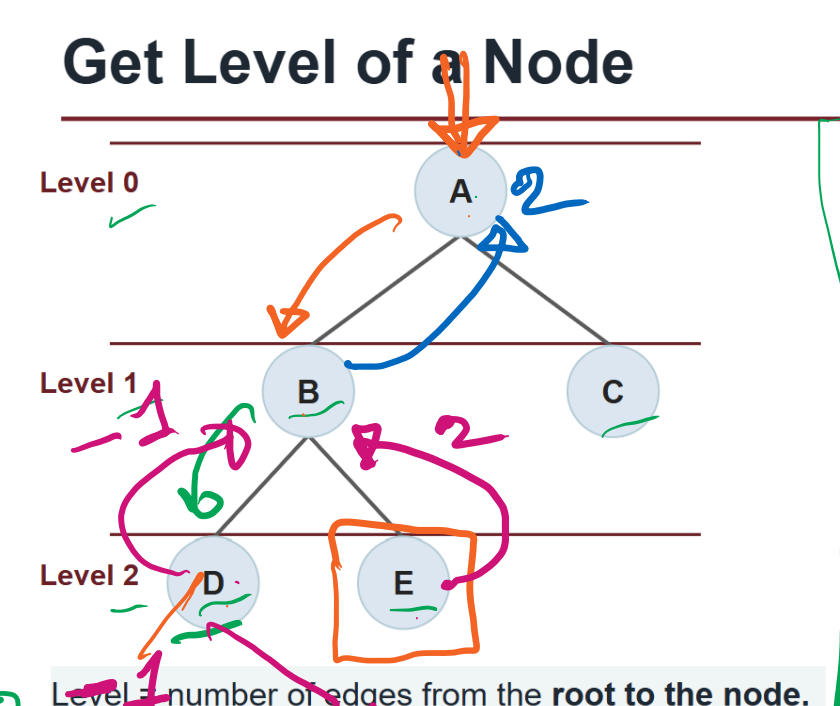


Index	0	1	2	3	4	5	6	7
Value	N	A	B	C	D	N	N	G

```
public static TNode arrToTree(String[] arr, int i) {
    // Base Case: Check if index is out of bounds or
    the current element is None
    if (i >= arr.length || arr[i] == null) {
        return null;
    }

    // Recursive Case:
    // 1. Create a node for the current index
    TNode root = new TNode(arr[i]);

    // 2. Set the left child by
          recursively calling for index 2 * i
    root.left  = arrToTree(arr, 2 * i);

    // 3. Set the right child by
          recursively calling for index 2 * i + 1
    root.right = arrToTree(arr, 2 * i + 1);

    return root;
}
# Driver part # arrToTree(arr, 1)
```

3 3 4 A

Get Level of a Node



Level = number of edges from the **root to the node.**

Start with level 0 at the root and increase by 1 in each recursive call.

Example:
level(E) = 2,
level(C) = 1,
level(P) = -1 (value not found).

current E E 2

```
public static int getLevel(TNode root, String value, int level){
    // Base case: value not found
    if (root == null) {
        return -1;
    }

    // If the current node matches the value, return the current level
    if (root.val.equals(value)) {
        return level;
    }

    // Recursive case: Search in the left subtree
    int leftLevel = getLevel(root.left, value, level + 1);
    // Recursive case: Search in the right subtree
    int rightLevel = getLevel(root.right, value,level + 1);

    // If value was not found in either subtree, return -1
    // Leaf node--one which has left and right node to be None
    if left_level == -1 and right_level == -1:
        return -1

    // Return the level where the value was found,
    //either in the left or right subtree (whichever is not -1)
    //--because -1 means not found in that sub tree
    return Math.max(leftLevel, rightLevel);
}
```

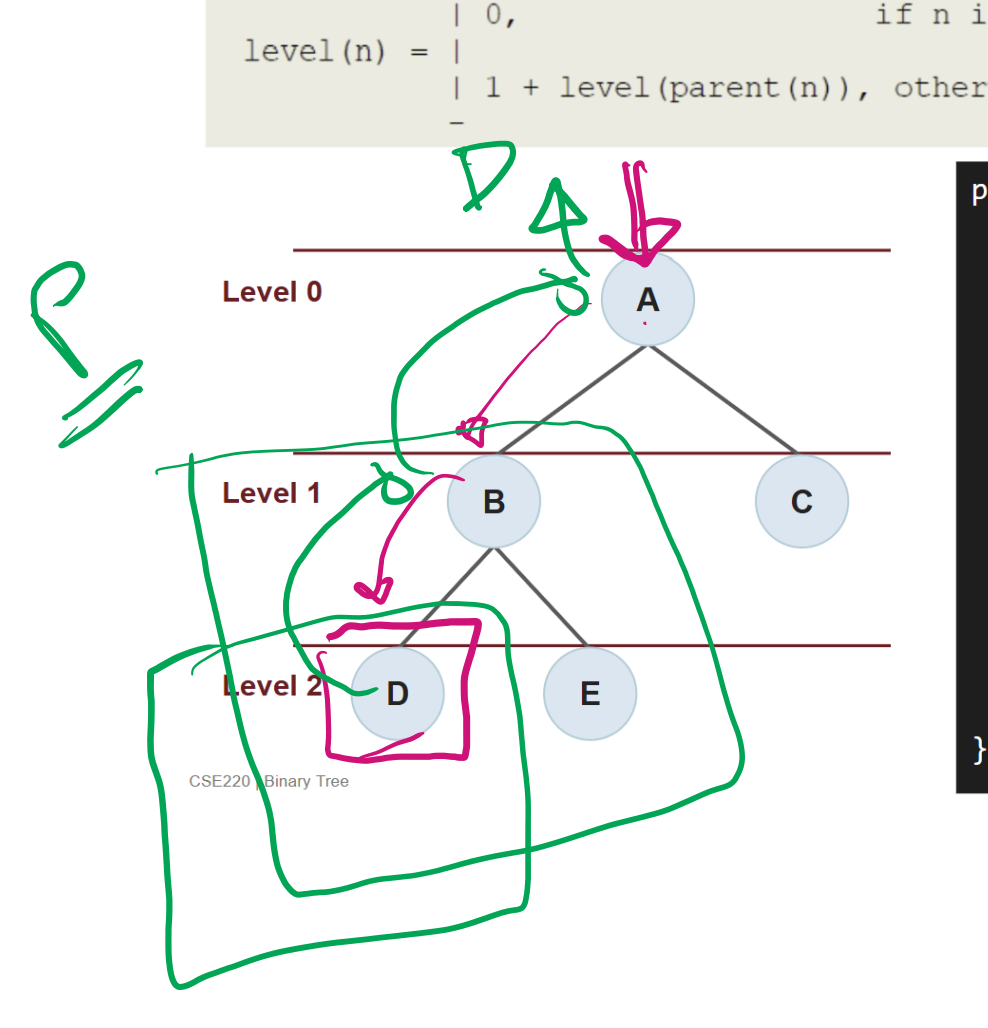
-1 2 2 -1 2 2

Finding the Node Before Computing Level with parent

First search the node by value. If found, compute the level by moving through parent pointers.

```
             _
            | 0,                    if n is the root (parent(n) == null)
level(n) =  |
            | 1 + level(parent(n)), otherwise
             _
```

current D

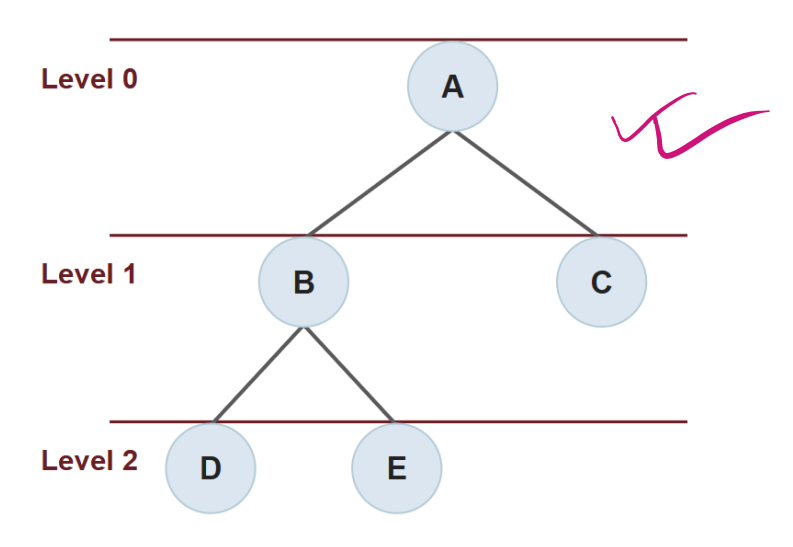


```
public static TPNode findNode(TPNode root, String target) {
    """ if node found will return the node, otherwise None"""

    // Base case: target not found
    if (root == null) return null;

    // Base case: Target found
    if (root.val.equals(target)) return root;

    // Search in the left subtree
    TPNode left = findNode(root.left, target);
    if (left != null) return left;

    // If not found in left subtree, search in the right subtree
    return findNode(root.right, target);
}
```

Finding the Node Before Computing Level with parent

First search the node by value. If found, compute the level by moving through parent pointers.

```
             _
            | 0,                    if n is the root (parent(n) == null)
level(n) =  |
            | 1 + level(parent(n)), otherwise
             _
```



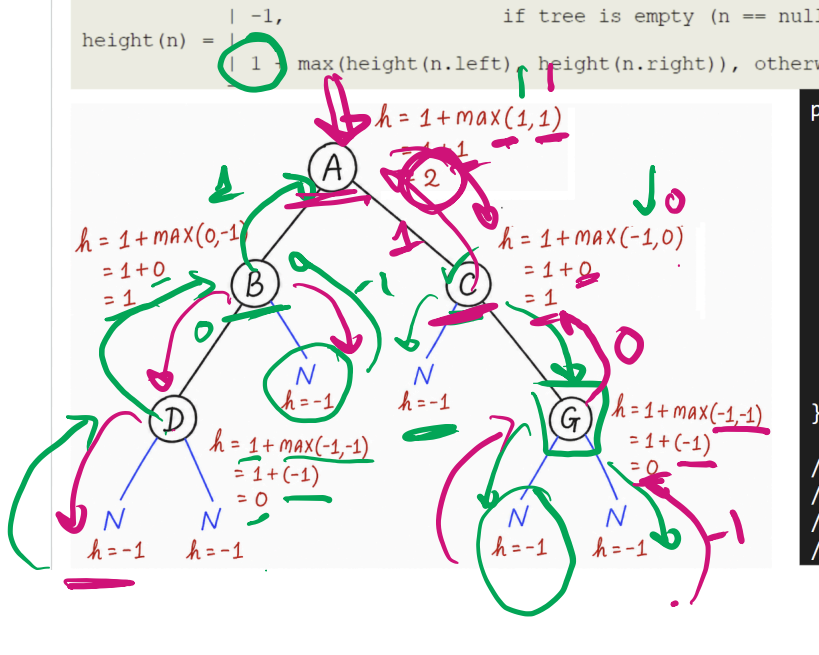
```
public static int getLevelWithParent(TPNode node) {
        // Base case: Target not found
        if (node == null) return -1;

        // Base case: Root node has level 0
        if (node.parent == null) return 0;

        // Recursive case
        return 1 + getLevelWithParent(node.parent);
  }


// Use together:
// TPNode node = findNode(root, "D");
// int level = getLevelWithParent(node);
```

N N P N -1

Get Height of a Node

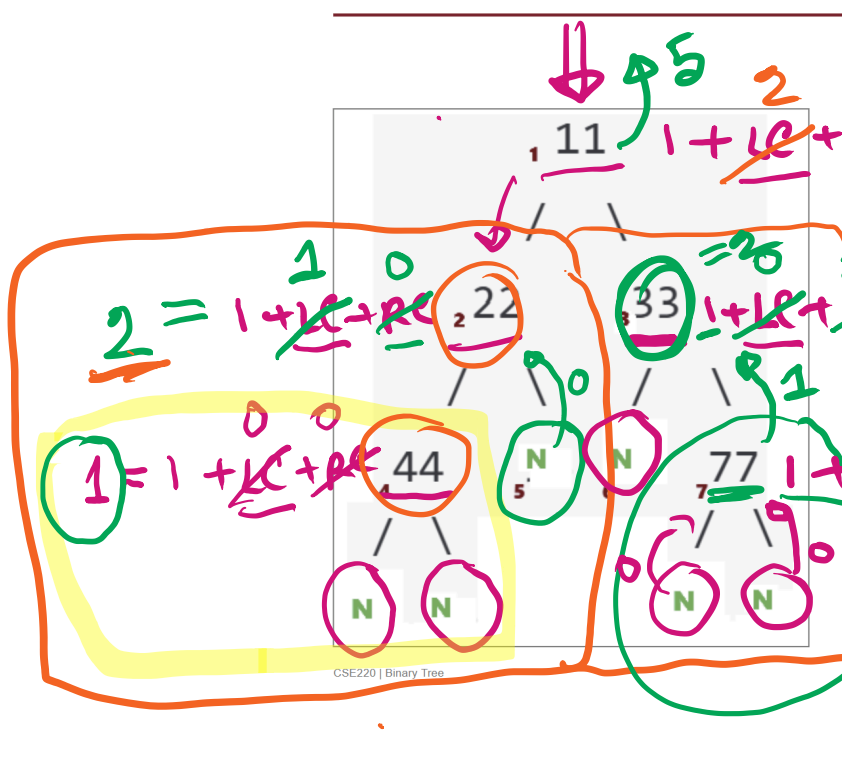
Height = length of the longest path from a node to one of its leaf nodes.
Empty subtree height is -1, so a leaf node has height 0.

```
             _
            | -1,                  if tree is empty (n == null)
height(n) = |
            | 1 + max(height(n.left), height(n.right)), otherwise
```



```
public static int getHeight(TNode node) {
    // Base Case: empty tree
    if (node == null) return -1;

    // Recursive Case:
    // Get the heights of the left and right subtrees
    int leftHeight = getHeight(node.left);
    int rightHeight = getHeight(node.right);

    // Compute the height of the current subtree
    return 1 + Math.max(leftHeight,rightHeight);
}

// Example:
// Height of A = 2
// Height of B, C = 1
// Height of leaf (D, G) = 0
```

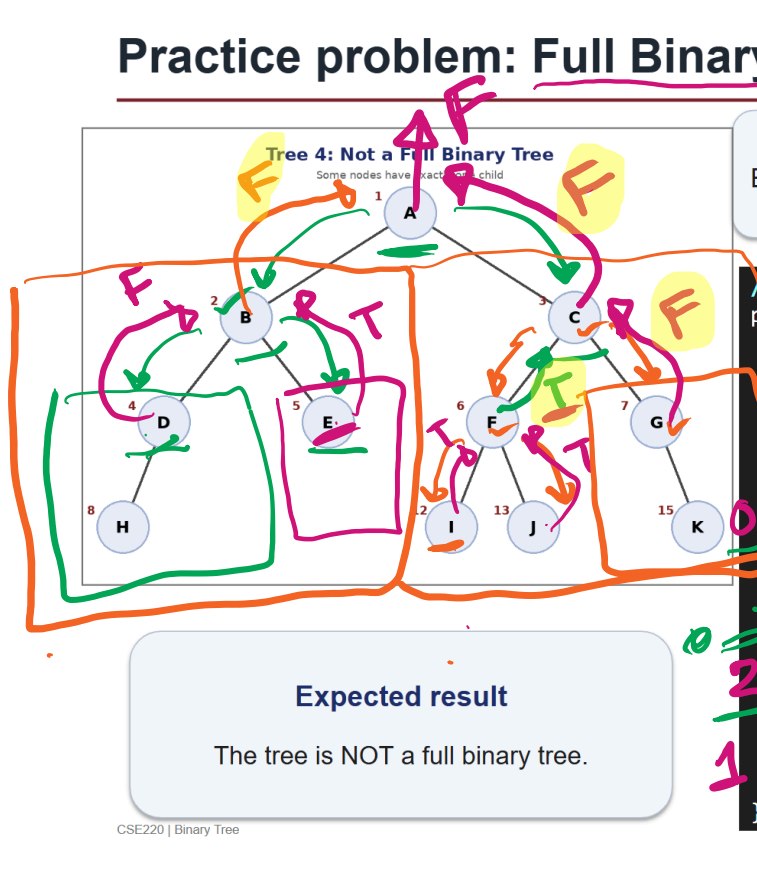
Get Size of a Binary Tree



Size = total number of **actual nodes** in the tree.
Null links are not counted as nodes.

```
public static int getSize(TNode node) {
    // Base case
    if (node == null) return 0;

    // Recursive case:
     The size is the current node (1)
     + size of left subtree + size of right subtree
    return 1 + getSize(node.left) + getSize(node.right);
}
// Output: Size of the tree: 5
```

Practice problem: Full Binary Tree Check


Tree 4: Not a Full Binary Tree
Some nodes have exactly one child

Expected result

The tree is NOT a full binary tree.

Rule of a full binary tree

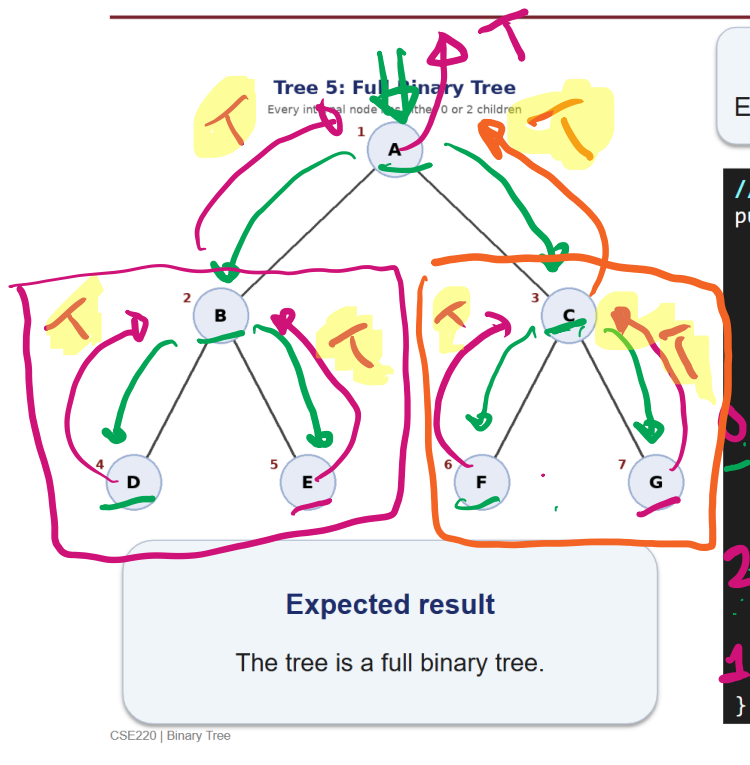
Every node has either exactly **0 children or exactly 2 children**.

leaf internal

```
// Returns true if the tree is a full binary tree
public static boolean isFullTree(TNode root) {
    // Empty tree is full
    if (root == null) {
        return true;
    }

    // Leaf node is valid
    if (root.left == null && root.right == null) {
        return true;
    }

    // Node with two children: both subtrees must also be full
    if (root.left != null && root.right != null) {
        return isFullTree(root.left) && isFullTree(root.right);
    }

    // Node with only one child: not full
    return false;
}
```

0 2 1 T T

Practice problem: Full Binary Tree Check


Tree 5: Full Binary Tree
Every internal node has either 0 or 2 children

Expected result

The tree is a full binary tree.

Rule of a full binary tree

Every node has either exactly **0 children or exactly 2 children**.

```
// Returns true if the tree is a full binary tree
public static boolean isFullTree(TNode root) {
    // Empty tree is full
    if (root == null) {
        return true;
    }

    // Leaf node is valid
    if (root.left == null && root.right == null) {
        return true;
    }

    // Node with two children: both subtrees must also be full
    if (root.left != null && root.right != null) {
        return isFullTree(root.left) && isFullTree(root.right);
    }

    // Node with only one child: not full
    return false;
}
```

Practice problem: Check whether the tree is Balanced

Balanced / Height-balanced Binary Tree:

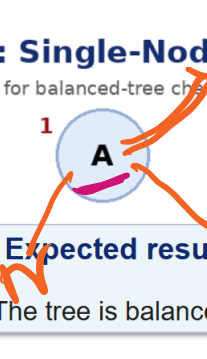
The **height** of the left and right subtree of any node differ by not more than 1.

df = | height(left) - height(right) |

Acceptable values of df = 0, 1
If df>1, then not balanced

Tree 1: Single-Node Tree

Used for balanced-tree checking



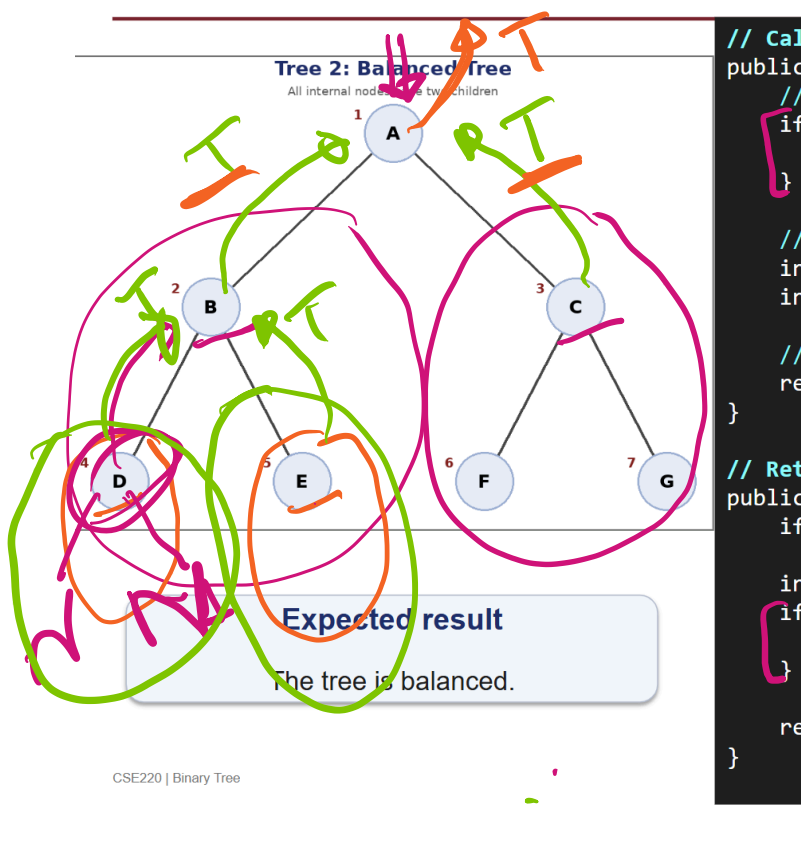
Expected result

The tree is balanced.

```
// Calculates the height difference between left and right subtrees
public static int getDf(TNode node) {
    // Base case: if node is null, return -1
    if (node == null) {
        return -1;
    }

    // Get the height of left and right subtrees
    int leftHeight = getHeight(node.left);
    int rightHeight = getHeight(node.right);

    // Return the absolute difference
    return Math.abs(leftHeight - rightHeight);
}

// Returns true if the tree is height-balanced
public static boolean isBalanced(TNode root) {
    if (root == null) return true;

    int df = getDf(root);
    if (df > 1) {
        return false;
    }

    return isBalanced(root.left) && isBalanced(root.right);
}
```

-1 -1 |-1-(-1)|=0 T T =T

Practice problem: Check whether the tree is Balanced


Tree 2: Balanced Tree
All internal nodes have two children

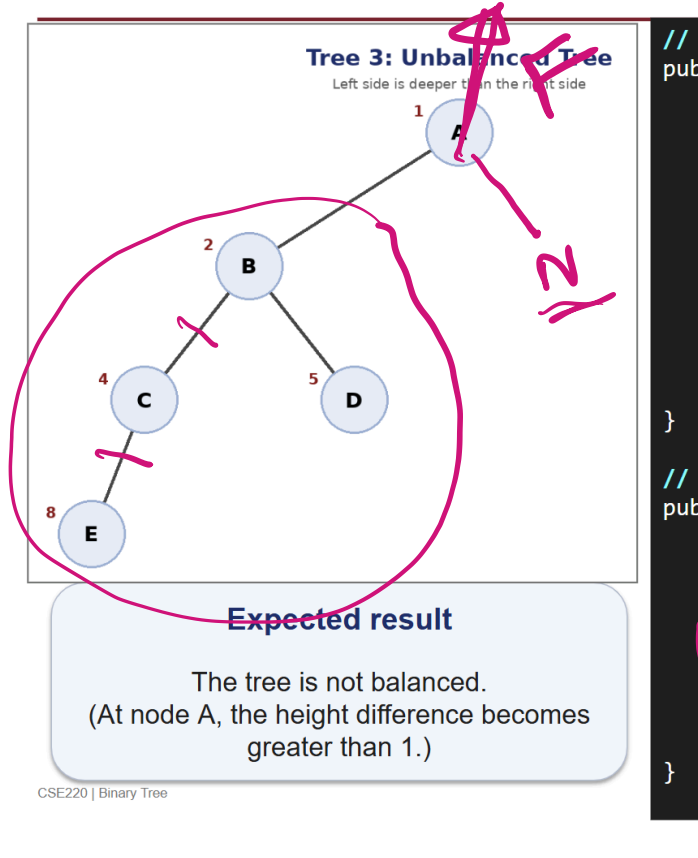
Expected result

The tree is balanced.

```
// Calculates the height difference between left and right subtrees
public static int getDf(TNode node) {
    // Base case: if node is null, return -1
    if (node == null) {
        return -1;
    }

    // Get the height of left and right subtrees
    int leftHeight = getHeight(node.left);
    int rightHeight = getHeight(node.right);

    // Return the absolute difference
    return Math.abs(leftHeight - rightHeight);
}

// Returns true if the tree is height-balanced
public static boolean isBalanced(TNode root) {
    if (root == null) return true;

    int df = getDf(root);
    if (df > 1) {
        return false;
    }

    return isBalanced(root.left) && isBalanced(root.right);
}
```

h = -1 h = -1 = -1-(-1) = 0 T N T N =T

Practice problem: Check whether the tree is Balanced


Tree 3: Unbalanced Tree
Left side is deeper than the right side

Expected result

The tree is not balanced.
(At node A, the height difference becomes greater than 1.)

```
// Calculates the height difference between left and right subtrees
public static int getDf(TNode node) {
    // Base case: if node is null, return -1
    if (node == null) {
        return -1;
    }

    // Get the height of left and right subtrees
    int leftHeight = getHeight(node.left);
    int rightHeight = getHeight(node.right);

    // Return the absolute difference
    return Math.abs(leftHeight - rightHeight);
}

// Returns true if the tree is height-balanced
public static boolean isBalanced(TNode root) {
    if (root == null) return true;

    int df = getDf(root);
    if (df > 1) {
        return false;
    }

    return isBalanced(root.left) && isBalanced(root.right);
}
```

height()

2 -1 |2-(-1)| = |2+1| = 3